

CS 4269/5469– Fundamentals of Logic In Computer Science

SEMESTER II, 2025-2026

Home Work-6

- You are welcome to consult any sources you find helpful—books, papers, websites, or even other people. However, you must clearly cite every source you use, and all submitted work must be written in your own words.
 - We strongly discourage the use of AI tools for this homework. The goal of the assignment is to help you engage deeply with the material discussed in class, and relying on AI tends to defeat this purpose. That said, if you do choose to use AI in any way, please acknowledge it and briefly explain how and where it was used. There will be no penalty for doing so; we are simply interested in understanding how these tools are being used.
- You may work either individually or in teams of two. Individual submissions receive a small bonus, but they are expected to require roughly twice the effort of a team submission. If you work in a team, please make sure that the workload is shared fairly.
- For team submissions, all members should be familiar with and have discussed the solutions to every question. This homework is meant to encourage independent thinking and discussion beyond what is covered in lectures.
- Please submit your solutions electronically as a PDF on Canvas. You are encouraged (but not required) to typeset your solutions in $\text{\LaTeX}$. Handwritten submissions are also acceptable.
- The assignment is due on April 20, 2026 (11:59 pm). The grading policy is as follows:
 - Submissions before April 17, 2026 (11:59 pm): 30% extra credit
 - Submissions from April 18, 2026 (12:00 am) to April 18, 2026 (11:59 pm): 15% extra credit
 - Submissions from April 20, 2026 (12:00 am) to April 20, 2026 (11:59 pm): no penalty
 - Submissions from April 21, 2026 (12:00 am) to April 21, 2026 (11:59 pm): 25% penalty
 - Submissions from April 22, 2026 (12:00 am) to April 22, 2026 (11:59 pm): 50% penalty
 - Submissions from April 23, 2026 (12:00 am) onwards: not accepted

Additionally, individual (non-team) submissions receive a 10% bonus applied on top of the above scoring scheme.

Computability

In the first part of this homework, you will be doing some warm-up exercises relevant to topics in computability.

Turing Machines Syntax. Let us first briefly recall the syntax of the Turing machine. We will work with single tape Turing machines¹. A Turing Machine is a tuple $M = (Q, q_{\text{init}}, q_{\text{acc}}, q_{\text{rej}}, \Gamma, \delta)$, where

- Q is the (finite) set of states of M .
- $q_{\text{init}} \in Q$ is the initial state of M .
- $q_{\text{acc}} \in Q$ is the unique accepting state of M .
- $q_{\text{rej}} \in Q$ is the unique rejecting state of M ; this is sometimes omitted.
- Γ is the (finite) tape alphabet. We assume that this also includes the blank symbol: $\sqcup \in \Gamma$. We will use $\Gamma_{\sqcup} = \Gamma \setminus \{\sqcup\}$ to denote the set of tape symbols other than the blank symbol.
- $\delta : Q \times \Gamma \times \Gamma \rightarrow Q \times \{L, R, S\} \times \Gamma \times \{L, R, S\}$ is the transition function of M .

Given a state $q \in Q$, tape symbol $\sigma_{\text{inp}} \in \Gamma$ at the input tape head and $\sigma_{\text{work}} \in \Gamma$ at the work tape head, the transition function returns a tuple $(q', m_{\text{inp}}, \sigma'_{\text{work}}, m_{\text{work}}) = \delta(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$. Based on this, the Turing machine changes its state from q to q' , moves the input tape head (resp. work tape head) left/right/nowhere depending upon whether m_{inp} (resp. m_{work}) is L, R or S after writes the symbol σ'_{work} at the cell corresponding to the current work tape head.

Accepting inputs. A Turing machine M halts on input $w \in \Gamma_{\sqcup}^*$ if the run of M on w reaches either the state q_{acc} or the state q_{rej} . An input w is accepted by M if the run (which maybe finite or infinite) of M on w reaches the state q_{acc} ; otherwise w is said to be *not accepted* by M . The language accepted by a Turing Machine M is $L(M) = \{w \mid w \text{ is accepted by } M\}$.

RE and REC languages. A language $L \subseteq \Gamma_{\sqcup}^*$ is said to be *recursively enumerable* if there is a Turing machine M such that $L = L(M)$. The set² of recursively enumerable languages is denoted **RE**. A language L is said to be *recursive* if there is a Turing machine M such that $L = L(M)$ and further, M halts on every input (whether or not that input is accepted or rejected by M). The class of recursive languages is called **REC**. The class **REC** is also the class of *decidable* languages (or decidable problems). A problem/language not in **REC** is said to be *undecidable*. A problem/language is said to be *semi-decidable* if it is in **RE** but not in **REC** (i.e., there is a Turing machine for the language, but no halting Turing machine.)

Computable Functions. Turing machines can also sometimes have a dedicated output tape, which is write-only. In this case, the transition function also describes what symbol to write

¹For every k -tape Turing machine M , there is a 1-tape Turing machine M' such that $L(M) = L(M')$

²Sets of languages are also sometimes referred to as *class* of languages, to avoid the confusion arising from the fact that languages themselves are sets (of strings)

on the current cell of the output tape and where to move the output tape head. We will skip formalizing this. Any such Turing Machine M defines a *partial function*³ as follows:

$$f_M(w) = \begin{cases} w' & \text{if } M \text{ halts on } w \text{ and writes } w' \text{ on the output tape} \\ \text{undef} & \text{otherwise (i.e., } M \text{ does not halt on } w) \end{cases}$$

A *total* function $f : \Gamma_{\cup}^* \rightarrow \Gamma_{\cup}^*$ is said to be a *computable function* if there is a Turing machine M such that M halts on every input and $f = f_M$.

Many-one Reductions and RE-hardness. A language $A \subseteq \Gamma_{\cup}^*$ *many-one* reduces to language $B \subseteq \Gamma_{\cup}^*$, denoted $A \leq_m B$, if there is a computable function $f : \Gamma_{\cup}^* \rightarrow \Gamma_{\cup}^*$ such that for every $w \in \Gamma_{\cup}^*$, we have the following: $w \in A$ iff $f(w) \in B$. In this case, the function f is called a *reduction* from A to B . A language L is said to be **RE-hard** if for every language $L' \in \mathbf{RE}$, we have $L' \leq_m L$. A language L is said to be **RE-complete** if $L \in \mathbf{RE}$ and L is **RE-hard**.

Now, consider the following languages:

$$\begin{aligned} \text{MP} &= \{ \langle M, w \rangle \mid w \in L(M) \} \\ \overline{\text{MP}} &= \{ \langle M, w \rangle \mid w \notin L(M) \} \\ \text{K} &= \{ \langle M \rangle \mid \langle M \rangle \in L(M) \} \\ \overline{\text{K}} &= \{ \langle M \rangle \mid \langle M \rangle \notin L(M) \} \end{aligned}$$

In the above, $\langle M, w \rangle$ (resp. $\langle M \rangle$) is an *encoding* of the pair (M, w) (resp. encoding of the machine M) as a string.

Question 1. Establish the following for the above languages:

1. $\text{MP} \in \mathbf{RE}$. (**Hint:** Define the Universal Turing machine and show that it accepts MP).
2. $\text{K} \in \mathbf{RE}$.
3. $\overline{\text{K}}$ is not in **RE**. (**Hint:** Use a *diagonalization* argument).
4. MP is **RE-hard**, and thus MP is **RE-complete**.
5. $\overline{\text{MP}}$ is not in **RE** via a many-one reduction from $\overline{\text{K}}$.

Church-Turing Theorem

In this section, we will work towards proving the undecidability of the validity problem for FOL. Let us first formally define the language of all valid formulae:

$$\text{VALID}_{\text{FOL}} = \{ \varphi \mid \varphi \text{ is a FOL formula and } \emptyset \models \varphi \}$$

Theorem 1 (Church and Turing). $\text{VALID}_{\text{FOL}}$ is **RE-hard** (and thus undecidable).

³A partial function $f : A \hookrightarrow B$ maps elements of A to elements of B ; not every element $a \in A$ maybe mapped to some element in B , but an element cannot be mapped to two different elements (i.e., $f(a) = b_1$ and $f(a) = b_2$ imply that $b_1 = b_2$). When an element $a \in A$ is not mapped to any element in B , we use the notation $f(a) = \text{undef}$.

In order to establish Theorem 1, we show that the language MP many-one reduces to the language $\text{VALID}_{\text{FOL}}$:

Lemma 1. $\text{MP} \leq_m \text{VALID}_{\text{FOL}}$.

In order to establish the reduction for Lemma 1, we show that there is a computable procedure to translate every string w to a well-formed FOL formula φ_w such that $w \in \text{MP}$ ⁴ iff $\varphi_w \in \text{VALID}_{\text{FOL}}$. The formula we construct will depend upon the word w and the Turing Machine M such that $\text{MP} = L(M)$; let this Turing Machine be $M = (Q, q_{\text{init}}, q_{\text{acc}}, q_{\text{rej}}, \Gamma, \delta)$.

Let us first describe our signature $\Sigma = (\mathcal{C}, \mathcal{F}, \mathcal{R})$. Our signature will use symbols that correspond to different parts of the Turing Machine. To distinguish elements of the signature from elements of the Turing machine, we will write the symbols in the signature in **red**.

- **Constant Symbols.** $\mathcal{C} = \{0\} \cup \{q \mid q \in Q\} \cup \{\gamma \mid \gamma \in \Gamma\}$.
 - The constant symbol **0** will be used to denote the number 0, which we will use to encode the initial configuration of the Turing machine.
 - For every state $q \in Q$, there is a constant symbol **q**. These will be used to denote the state of the Turing machine at each step of its execution.
 - For every tape symbol $\gamma \in \Gamma$, we have a constant symbol **γ**.
- **Function Symbols.** $\mathcal{F} = \{\mathbf{s}, \mathbf{state@}, \mathbf{inpHead@}, \mathbf{inpCell}, \mathbf{workHead@}, \mathbf{workCell}\}$.
 - The function symbol **s** is the *successor* function and $\text{arity}(f) = 1$. Informally, the expresion ‘ $y = \mathbf{s}(x)$ ’ will mean ‘ $y = x + 1$ ’. We need this function symbol to encode successively increasing time steps in the execution of the Turing machine.
 - The function symbol **state@** will be used to denote the state of the Turing Machine at any particular time step; $\text{arity}(f) = 1$. Informally, we will write ‘ $\mathbf{state@}(t) = \mathbf{q}$ ’ when we want to say that at the t^{th} step, the state of the Turing Machine is the state corresponding to the constant symbol **q**. We will add constraints to ensure that the range of this function is limited to the set of states.
 - The function symbols **inpHead@** and **workHead@** will be used to denote the position of the head of the input and work tapes; $\text{arity}(f) = \text{arity}(f) = 1$. Informally, we will write ‘ $\mathbf{inpHead@}(t) = i$ ’ (resp. ‘ $\mathbf{workHead@}(t) = i$ ’) to denote that at the t^{th} step, the head of the input tape (resp. work tape) is at position i .
 - The function symbols **inpCell** and **workCell** will be used to denote the contents of a particular cell of the input and work tapes; $\text{arity}(f) = \text{arity}(f) = 2$. Informally, we will write ‘ $\mathbf{inpCell}(t, i) = \gamma$ ’ (resp. ‘ $\mathbf{inpCell}(t, i) = \gamma$ ’) to denote that at the t^{th} step, the cell at position i of the input tape (resp. work tape) contains the tape symbol corresponding to the constant **γ**. We will add constraints to ensure that the range of the function is limited to the set of tape symbols Γ .
- **Relation Symbols.** We do not need any relation symbols for our encoding, so $\mathcal{R} = \emptyset$ suffices.

⁴Technically, our reduction is structured so that, in fact, we could have started with any language L in **RE** (and the corresponding Turing Machine M_L for L), instead of the very specific **RE** language MP. This is not surprising since the proof of **RE**-hardness of MP is pretty straightforward and directly lends itself to a proof of **RE**-hardness of $\text{VALID}_{\text{FOL}}$ via a reduction from any arbitrary **RE** language.

The formula φ_w will be a sentence (no free variables) and will have the following form:

$$\varphi_w \equiv \underbrace{(\varphi_{\text{successor}} \wedge \varphi_{\text{consistent}} \wedge \varphi_{\text{init}} \wedge \varphi_{\text{transition}})}_{\varphi_{\text{precondition}}} \rightarrow \varphi_{\text{accept}}$$

Informally, if φ_w is valid, then for every structure $\mathcal{A}$ that satisfies the properties demanded by $\varphi_{\text{precondition}}$, $\mathcal{A}$ must make the consequent φ_{accept} true. Our correctness proof will argue that indeed this happens whenever $w \in \text{MP}$ (and thus in this case φ_w will hold on all structures) and, conversely, this will fail to happen when $w \notin \text{MP}$. The formula $\varphi_{\text{precondition}}$ in this sense, abstracts away the reasoning about irrelevant structures — if structure $\mathcal{A}$ does not satisfy $\varphi_{\text{precondition}}$, then $\mathcal{A} \models \varphi_{\text{precondition}}$ and thus the only structures that we need to reason about will be the ones that are *interesting*: those that have all the preproperties demanded by $\varphi_{\text{precondition}}$.

Let us describe each of the sub-formulae $\varphi_{\text{successor}}$, $\varphi_{\text{consistent}}$, φ_{init} , $\varphi_{\text{transition}}$ and φ_{accept} below. Our encoding ensures that each of these sub-formulae are sentences.

Successor. In our formula, we will encode the run of a Turing machine. For this, we need the notion of Natural Numbers $\mathbb{N}$, so that we can say "at the i^{th} step, the configuration of the Turing machine is so-and-so". Since there is no a-priori bound on the length of the run of the Turing machine, we would like our formula to This means that we would like to ensure that a part of our universe looks like the Natural numbers: assuming the constant $\mathbf{0}$ behaves like the number $0 \in \mathbb{N}$, we would like that the function $\mathbf{s}$ behaves as the '+1' function for natural numbers. We say this as follows:

$$\varphi_{\text{successor}} \equiv \left(\forall x \cdot \neg(\mathbf{0} = \mathbf{s}(x)) \right) \quad \wedge \quad \left(\forall x, y \cdot (\mathbf{s}(x) = \mathbf{s}(y)) \rightarrow (x = y) \right)$$

The first conjunct in the above formula states that there is no $\mathbf{s}$ -predecessor of the element corresponding to the constant $\mathbf{0}$. The second conjunct says that the same element cannot not have two $\mathbf{s}$ -predecessors. These two together ensure that there is a subset of the universe that looks like the natural numbers. Let us first formally establish this fact.

Lemma 2. Let $\mathcal{A} = (U, \mathcal{I})$ be a Σ -structure such that $\mathcal{A} \models \varphi_{\text{successor}}$. Then, there is a subset $U' \subseteq U$ that is isomorphic to $\mathbb{N}$. That is, $U' = \{u_0, u_1, \dots\}$ (where each u_i s are distinct), $u_0 = \mathcal{I}(\mathbf{0})$, and for each $i \in \mathbb{N}$, $u_i = \mathcal{I}(\underbrace{\mathbf{s}(\mathbf{s}(\dots \mathbf{s}(\mathbf{0}) \dots)}_{i \text{ times}}))$.

Question 2. Prove Lemma 2.

Consistency. The formula $\varphi_{\text{consistent}}$ talks about some basic consistency properties that we expect our structures to satisfy. Namely, all constants $\{\mathbf{q} \mid q \in Q\}$ are interpreted to be distinct elements, all constants $\{\gamma \mid \gamma \in \Gamma\}$ are interpreted to be distinct elements and the range of state@ , inpCell and workCell is limited to a few constants.

$$\begin{aligned}
\varphi_{\text{consistent}} \equiv & \bigwedge_{q \neq q' \in Q} \mathbf{q} \neq \mathbf{q}' \\
& \wedge \bigwedge_{\gamma \neq \gamma' \in \Gamma} \mathbf{\gamma} \neq \mathbf{\gamma}' \\
& \wedge \forall t \cdot \left(\bigvee_{q \in Q} \mathbf{state@}(t) = \mathbf{q} \right) \\
& \wedge \forall t, i \cdot \left(\bigvee_{\gamma \in \Gamma} \mathbf{inpCell}(t, i) = \mathbf{\gamma} \right) \\
& \wedge \forall t, i \cdot \left(\bigvee_{\gamma \in \Gamma} \mathbf{workCell}(t, i) = \mathbf{\gamma} \right)
\end{aligned}$$

Initial Configuration. Here, we need to encode the initial configuration of M . In the beginning, the state is q_0 , the work tape only has blank symbols, the input tape has the input word w written on it and the tape heads are at the 0th cell.

$$\begin{aligned}
\varphi_{\text{init}} \equiv & \mathbf{state@}(0) = \mathbf{q_0} \wedge \mathbf{inpHead@}(0) = \mathbf{0} \wedge \mathbf{workHead@}(0) = \mathbf{0} \\
& \wedge \bigwedge_{i=0}^{|w|-1} (\mathbf{inpCell}(0, \mathbf{s}^i(0)) = \mathbf{w}[i]) \\
& \wedge \left(\forall x \cdot \left(\bigwedge_{x=0}^{|w|-1} \neg(x = \mathbf{s}^i(0)) \right) \rightarrow (\mathbf{inpCell}(0, x) = \mathbf{\sqcup}) \right) \\
& \wedge \forall x \cdot \mathbf{workCell}(0, x) = \mathbf{\sqcup}
\end{aligned}$$

Transition. The encoding of the transition function is a conjunction of different formulae, one corresponding to every tuple of the form (q, σ, σ') :

$$\varphi_{\text{transition}} \equiv \cdot \bigwedge_{\substack{q \in Q \\ \sigma_{\text{inp}} \in \Gamma \\ \sigma_{\text{work}} \in \Gamma}} \varphi(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$$

The sub-formulae $\varphi(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$ encode the following statement : if at any time t , the state is q and the symbols corresponding to the input and work tape heads are σ_{inp} and σ_{work} respectively, then at the time $t + 1$, the next state, the symbol written at the current work tape head and the positions of the heads of the two tapes must be according to the output of the transition $(q', m_{\text{inp}}, \sigma'_{\text{work}}, m_{\text{work}}) = \delta(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$. In the following, we write this formula for a fixed $(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$ and thus we fix the output of the transition $(q', m_{\text{inp}}, \sigma'_{\text{work}}, m_{\text{work}})$. Thus, $\varphi(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$ has the form

$$\varphi(q, \sigma_{\text{inp}}, \sigma_{\text{work}}) \equiv \forall t \cdot \left(\begin{array}{l} \mathbf{state@}(t) = \mathbf{q} \\ \wedge \mathbf{inpCell}(t, \mathbf{inpHead@}(t)) = \mathbf{\sigma_{inp}} \\ \wedge \mathbf{workCell}(t, \mathbf{workHead@}(t)) = \mathbf{\sigma_{work}} \end{array} \right) \rightarrow \psi(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$$

The formula $\psi(q, \sigma_{\text{inp}}, \sigma_{\text{work}})$ encodes the facts that (a) the state at time $t + 1$ is q' , (b) the input head at time $t + 1$ is determined by m_{inp} , (c) the contents of the input tape at $t + 1$ is the same as at time t , (d) the work tape head at time $t + 1$ moves according to m_{work} , and (e) the work tape contents change at $t + 1$, with the only change being at the current cell, based on what σ'_{work} is.

The variable t occurs free in this formula.

$$\begin{aligned} \psi(q, \sigma_{\text{inp}}, \sigma_{\text{work}}) \equiv & \text{state@}(\mathbf{s}(t)) = \mathbf{q}' \\ & \wedge \alpha_{m_{\text{inp}}} \wedge \beta_{m_{\text{work}}} \\ & \wedge \forall x \cdot \text{inpCell}(\mathbf{s}(t), x) = \text{inpCell}(t, x) \\ & \wedge \text{workCell}(\mathbf{s}(t), \text{workHead@}(t)) = \sigma'_{\text{work}} \\ & \wedge \forall x \cdot \neg(x = \text{workHead@}(t)) \rightarrow (\text{workCell}(\mathbf{s}(t), x) = \text{workCell}(t, x)) \end{aligned}$$

Here the formulae $\{\alpha_m\}_{m \in \{L, S, R\}}$ and $\{\beta_m\}_{m \in \{L, S, R\}}$ relate the position of the input and work tape heads at time t with their positions at time $t + 1$.

$$\alpha_m \equiv \begin{cases} \exists x \cdot \mathbf{s}(x) = \text{inpHead@}(t) \wedge \text{inpHead@}(\mathbf{s}(t)) = x & \text{if } m = L \\ \text{inpHead@}(\mathbf{s}(t)) = \text{inpHead@}(t) & \text{if } m = S \\ \text{inpHead@}(\mathbf{s}(t)) = \mathbf{s}(\text{inpHead@}(t)) & \text{otherwise if } m = R \end{cases}$$

$$\beta_m \equiv \begin{cases} \exists x \cdot \mathbf{s}(x) = \text{workHead@}(t) \wedge \text{workHead@}(\mathbf{s}(t)) = x & \text{if } m = L \\ \text{workHead@}(\mathbf{s}(t)) = \text{workHead@}(t) & \text{if } m = S \\ \text{workHead@}(\mathbf{s}(t)) = \mathbf{s}(\text{workHead@}(t)) & \text{otherwise if } m = R \end{cases}$$

Accepting Condition. This is the simplest formula. It says that at some (finite) time, the Turing Machine reaches the accept state:

$$\varphi_{\text{accept}} \equiv \exists t \cdot (\text{state@}(t) = \mathbf{q}_{\text{acc}})$$

Proof of Correctness of Reduction

To prove that the reduction is correct, we need to establish that (1) it can be effectively computed, (2) it is sound (i.e., if $\varphi_w \in \text{VALID}_{\text{FOL}}$, then $w \in \text{MP}$), and (3) it is complete (i.e., if $w \in \text{MP}$, then $\varphi_w \in \text{VALID}_{\text{FOL}}$).

Question 3. Argue that the reduction is a computable function.

Let us now proceed towards the easier direction (soundness).

Soundness. We assume that φ_w is valid. Starting with this assumption, we will show that w has an accepting run in the Turing machine M . First, since φ_w is valid, it must hold in every structure. In particular, for the structure $\mathcal{A}_{\mathbb{N}, M} = (U_{\mathbb{N}, M}, \mathcal{I}_{\mathbb{N}, M})$ that we describe next, we must have $\mathcal{A}_{\mathbb{N}, M} \models \varphi_w$. The universe of the structure $\mathcal{A}_{\mathbb{N}, M}$ is the set of natural numbers. In the following, you are asked to provide a reasonable definition of the interpretation that helps you complete the rest of the soundness proof.

Question 4. Complete the description of the interpretation $\mathcal{I}_{\mathbb{N}, M}$ by specifying it for every constant and function symbols in the signature Σ .

Question 5. Complete the proof of soundness of the reduction by arguing that the interpretation constructed above can be used to construct a run of the Turing machine M on w that is also an accepting run.
(Hint: You may want to use induction to argue that each prefix of the run obeys the semantics of the Turing machine M .)

Completeness. Here, we start with the assumption that $w \in \text{MP}$ and argue that φ_w is valid, or equivalently, for every structure $\mathcal{A}$, we have $\mathcal{A} \models \varphi_w$. First, if $\mathcal{A}$ does not satisfy one or more of $\varphi_{\text{successor}}$, $\varphi_{\text{consistent}}$, φ_{init} , $\varphi_{\text{transition}}$, then clearly $\mathcal{A} \models \varphi_w$. So we need to only argue about those structures $\mathcal{A}$ for which each of $\mathcal{A} \models \varphi_{\text{successor}}$, $\mathcal{A} \models \varphi_{\text{consistent}}$, $\mathcal{A} \models \varphi_{\text{init}}$ and $\mathcal{A} \models \varphi_{\text{transition}}$ hold. Clearly, any such model must have a sub-universe that looks like the natural number line (Lemma 2). However, the challenge is that it may have more elements in addition this sub-universe. This is because we need to, in principle, reason about all elements of the universe (and not just our special sub-universe). Thankfully, the existential quantification (over t) in the consequent φ_{accept} allows us to only reason about a subset of the elements of any such structure. Indeed, consider the hypothetical reduction attempt that reduces instead from the complement language $\overline{\text{MP}}$ by constructing the formula

$$\varphi'_w \equiv (\varphi_{\text{successor}} \wedge \varphi_{\text{consistent}} \wedge \varphi_{\text{init}} \wedge \varphi_{\text{transition}}) \rightarrow (\forall t \cdot \neg(\text{state@}(t) = q_{\text{acc}}))$$

The above formula will not hold in structures where there are elements apart from the set of Natural numbers, since $\text{state@}(t)$ might point to q_{acc} for some t that is not reachable from the number 0 with successive applications of s .

The key to showing that $\mathcal{A} \models \varphi_{\text{accept}}$ is to argue that the interpretation to the function symbols must simulate the run of the Turing machine on w . Let the sequence of configurations of M on w be $c_0, c_1, \dots, c_k$ where c_k is the accepting configuration (i.e., the state corresponding to c_k is q_{acc}). We need to argue that the fact that $\mathcal{A} \models \varphi_{\text{precondition}}$ ensures that the interpretation of the function symbols inpCell , inpHead@ , workCell and workHead@ is consistent with the run, i.e., for each $t \in \mathbb{N}$, the interpretation of $\text{workHead@}(t)$ matches that of the configuration c_t . This argument will follow using an induction on the length of the sequence of configurations. We leave the details to the reader. Having proved this, it follows that the sub-universe carved out using the terms $\{s^i(0)\}_i$, behaves like the natural number line, and further, for the element corresponding to the term $\{s^i(0)\}_k$ corresponding to the number k , it holds that $\mathcal{I}(\text{state@})(k) = \mathcal{I}(q_{\text{acc}})$, and thus $\mathcal{A} \models \varphi_{\text{accept}}$.

Question 6. Prove completeness of the reduction by formally establishing the above argument. That is, formally show that if $w \in \text{MP}$, then φ_w is a valid FOL formula.

We sum it all up in the following.

Question 7. Prove Theorem 1

A Resolution Proof System for FOL

As with propositional logic, a resolution-style proof system has also been defined for proving unsatisfiability of FOL formulae. This was first formalized by J.A. Robinson⁵. We will recall this proof system in modern terminology and also prove its soundness and completeness using the results we have already learnt in the course so far. As with propositional logic, the resolution proof system for FOL is also catered towards establishing unsatisfiability of a set of formulae Γ (instead of establishing validity as in Hilbert style proof systems) over some signature Σ .

Simplifying assumptions. We will assume that Γ is a countable set of formulae, each of which is presented in prenex normal form. Also, we will assume that all formulae are universally quantified sentences. Further, we will assume that the formulae do not contain the ‘ $\equiv$ ’ symbol. Finally, we assume that the matrix of every formula $\varphi \in \Gamma$ is a clause, i.e., $\varphi = \forall x_1 \forall x_2 \cdots \forall x_k \cdot \left(\bigvee_{i=1}^n \alpha_i \right)$, where each α_i is an atomic formula, with $\text{FVar}(\alpha_i) \subseteq \{x_1, \dots, x_k\}$. In Homework-4 we have already formally established how to relax these assumptions when checking for unsatisfiability, and we skip repeating them here.

Let us now define what is a proof in this proof system. In addition to the usual resolution proof step of propositional logic, proofs here also have *instantiation* steps.

Definition 1 (Resolution Proof for FOL). A sequence $\pi = \varphi_1, \varphi_2, \dots, \varphi_n$ of sentences over a signature Σ is said to be a resolution proof for a set of universally quantified sentences Γ (over the same signature Σ) if $\varphi_n = \{\}$ is the empty clause, and for every $1 \leq i \leq n$, one of the following holds:

1. **(Axiom).** $\varphi_i \in \Gamma$.
2. **(Instantiation).** There is a $j < i$ such that $\varphi_i \in \text{Ground}(\varphi_j)$.
3. **(Resolvent).** There are $j, k < i$ such that φ_j and φ_k are ground clauses, and there is a ground atom α such that $\alpha \in \varphi_j$ and $\neg\alpha \in \varphi_k$ and $\varphi_i = (\varphi_j \setminus \{\alpha\}) \cup (\varphi_k \setminus \{\neg\alpha\})$.

Theorem 2. The resolution proof system defined above is sound. That is, let Γ be a set of universally quantified sentences, with the assumptions outlined above, over signature Σ . If Γ has a resolution proof, then Γ is unsatisfiable.

Question 8. Prove Theorem 2

Hint: Instead of proving soundness from scratch, use Herbrand’s Theorem, together with the soundness of propositional resolution proof system.

Theorem 3. The resolution proof system defined above is complete. That is, let Γ be a set of universally quantified sentences, with the assumptions outlined above, over signature Σ . If Γ is unsatisfiable, then there is a resolution proof for Γ .

⁵J. A. Robinson. 1965. A Machine-Oriented Logic Based on the Resolution Principle. J. ACM 12, 1 (Jan. 1965), 23–41. <https://doi.org/10.1145/321250.321253>

Question 9. Prove Theorem 3

Hint: Instead of proving soundness from scratch, use Herbrand's Theorem, together with the completeness of propositional resolution proof system.

Question 10. Use the soundness and completeness of the FOL resolution proof system to show that $\text{VALID}_{\text{FOL}} \in \text{RE}$ and thus $\text{VALID}_{\text{FOL}}$ is **RE**-complete.

Small Model Property and FOL Fragments

In the last question, we have proved that the validity problem for FOL is **RE**-complete, thus undecidable. Its dual problem, the satisfiability problem for FOL, SAT_{FOL} , is **coRE**-complete, which is also undecidable. In this part, we focus on FOL fragments for which the classical decision problems, SAT_{FOL} and $\text{VALID}_{\text{FOL}}$, are decidable.

Before exploring certain FOL fragments, we first introduce a useful tool to prove decidability.

Definition 2 (Small Model Property). A class of $\mathcal{C}$ of FOL has the *small model property* if there is a computable function f such that for every $\phi \in \mathcal{C}$, if ϕ is satisfiable then it has a model of size at most $f(|\phi|)$.

The first FOL fragment is the class of quantifier-free FOL formulae.

Question 11. Formally define the quantifier-free fragment of FOL and show that it has the small model property.

Now we can prove that its classical decision problems are decidable.

Question 12. Prove that the satisfiability and validity problems for quantifier-free fragment of FOL are decidable.

The second FOL fragment is **EPR** (or Bernays-Schönfinkel) class of FOL formulae, containing function-free formulae with quantifiers of the form $\exists \dots \exists \forall \dots \forall$ (or $\exists^* \forall^*$).

Question 13. Prove that the satisfiability problem for **EPR** fragment of FOL is decidable.